

HANDS-ON PARALLEL PROGRAMMING

OpenMP Programming

A Practical Tutorial in C

From your first thread to performance analysis

English edition • Linux laptop or one Linux virtual machine

What you will build

Eight small programs that demonstrate threads, data sharing, parallel loops, safe accumulation, loop scheduling, synchronization, tasks, and a numerical benchmark. Each exercise includes a complete program, commands, expected behavior, and questions to investigate.

Learning outcomes

After completing the tutorial, you should be able to:

1. Explain how a team of OpenMP threads executes a parallel region.
2. Compile and run OpenMP programs, and control the requested thread count.
3. Identify independent loop iterations and choose data-sharing attributes.
4. Recognize data races and use reductions or synchronization correctly.
5. Compare loop schedules and distinguish worksharing from task creation.
6. Validate numerical results and measure speedup using a fair baseline.

Prerequisites and scope

You should know basic C: variables, arrays, loops, functions, pointers, and compilation. The commands assume a Bash terminal on Ubuntu or a similar Debian-based Linux system. A Linux VM is sufficient; allocate at least two virtual CPUs if the host has spare CPU capacity. Four GB of guest RAM is comfortable for these examples. One CPU is enough to study correctness, but offers little opportunity for parallel speedup.

The tutorial uses CPU threads on one machine. OpenMP also includes accelerator features, which are outside this introductory lab. The source is self-contained: upload this `.tex` file to Overleaf and select pdfLaTeX. Compile the C programs in your Linux terminal; compiling the document does not execute them.

How to work through the exercises

Save each complete C listing using the filename shown above it. Type or copy code from the LaTeX source to avoid importing PDF line numbers. First compile and run the unchanged example, then perform the requested modifications. Keep the original working version so that you can compare behavior.

Contents

1	The OpenMP execution model	3
1.1	Process, thread, and core	3
1.2	Parallel regions and worksharing	3
1.3	Where OpenMP fits relative to MPI	3
2	Prepare the environment	4
2.1	Install a C compiler	4
2.2	Use a consistent build command	4
2.3	Request a thread count	4
3	Exercise 1: Create a team of threads	5
4	Exercise 2: Understand shared and private data	6
5	Exercise 3: Parallelize vector addition	7
6	Exercise 4: Fix accumulation with a reduction	9
7	Exercise 5: Compare loop schedules	11
8	Exercise 6: Coordinate dependent phases	12
9	Exercise 7: Create independent tasks	14
10	Exercise 8: Measure numerical correctness and speedup	15
11	Optional mini-project: Parallel matrix multiplication	19
12	Troubleshooting and good practice	20
12.1	A practical review checklist	20
13	Suggested lab report	21
	References and further reading	22

1 The OpenMP execution model

OpenMP is an application programming interface for parallel programming in C, C++, and Fortran. Its CPU programming model lets threads in one process access shared memory. Its interface includes compiler directives, runtime functions, and environment variables [1].

1.1 Process, thread, and core

- A **process** is a running program with its own address space.
- A **thread** is an execution stream inside a process. Threads have their own execution state while accessing the process's shared data.
- A **CPU core** is a hardware execution resource. A logical CPU may represent a hardware thread rather than a separate physical core.

Requesting four OpenMP threads does not create four physical cores. The operating system schedules software threads onto the CPU resources available to the process. In a VM, virtual CPUs also depend on host scheduling.

1.2 Parallel regions and worksharing

An ordinary program begins with one thread. When that thread encounters `#pragma omp parallel`, a team executes the region. The encountering thread participates as thread 0. At the region's end, the team synchronizes and the encountering thread continues after the region [2]. This is commonly described as a fork-join model; an implementation may reuse worker threads across regions.

Listing 1: A structural example, not a complete program

```

1  /* Executed by the initial thread. */
2  #pragma omp parallel
3  {
4      /* Every thread in the team executes this block. */
5  }
6  /* The initial thread continues here. */

```

Creating a team does not divide a loop automatically

`parallel` starts a parallel region. `for` distributes loop iterations among an existing team. `parallel for` combines those two actions. An ordinary loop inside a `parallel` region is executed in full by every thread unless you add an appropriate worksharing construct.

1.3 Where OpenMP fits relative to MPI

Aspect	OpenMP in this tutorial	Typical MPI program
Execution units	Threads in one process	Processes identified by ranks
Data access	Shared address space, with private data where needed	Separate process address spaces; communication is explicit
Hardware scope	One shared-memory machine	One machine or multiple machines
Hybrid use	Threads parallelize work within a node	MPI moves data between nodes in a hybrid MPI+OpenMP program

2 Prepare the environment

2.1 Install a C compiler

Run these commands in your Ubuntu terminal:

```
sudo apt update
sudo apt install build-essential
gcc --version
nproc
lscpu
mkdir -p ~/openmp-lab
cd ~/openmp-lab
```

`build-essential` includes GCC and basic build tools. With GCC, the `-fopenmp` option enables OpenMP directives and links the OpenMP runtime automatically [1]. You do not need an MPI installation for these programs.

`nproc` is a useful starting point for the CPUs visible to the current process. Its result is not a promise of dedicated CPU capacity. Use `lscpu` and your VM settings to record the hardware configuration.

2.2 Use a consistent build command

```
gcc -std=c11 -O2 -Wall -Wextra -fopenmp program.c -o program
./program
```

Option	Meaning
<code>-std=c11</code>	Compile as C11.
<code>-O2</code>	Enable compiler optimization. Keep this the same in comparisons.
<code>-Wall -Wextra</code>	Enable useful compiler warnings.
<code>-fopenmp</code>	Enable and link OpenMP.
<code>-o program</code>	Name the executable <code>program</code> .
<code>-lm</code>	Link the math library when a later example uses math functions.

2.3 Request a thread count

```
export OMP_DYNAMIC=FALSE
export OMP_NUM_THREADS=4
./program

# Change the request for one command only:
OMP_NUM_THREADS=2 ./program
```

`OMP_NUM_THREADS` specifies a requested team size. `OMP_DYNAMIC=FALSE` disables the runtime's dynamic adjustment of that size; other limits may still apply. Always inspect the actual team size *inside* a parallel region. The setting persists only in the current terminal and its child processes.

Start small

Use 1, 2, and 4 threads when those counts fit your machine or VM. Begin with the supplied problem sizes. Increasing the thread count beyond available CPU capacity can increase overhead rather than improve performance.

3 Exercise 1: Create a team of threads

Goal: distinguish the serial part of a program from its parallel region.

Listing 2: Save as `hello_omp.c`

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 int main(void)
5 {
6     printf("Before: team size = %d\n", omp_get_num_threads());
7
8     #pragma omp parallel default(none)
9     {
10         int tid = omp_get_thread_num();
11         int team_size = omp_get_num_threads();
12
13         #pragma omp critical
14         {
15             printf("Hello from thread %d of %d\n", tid, team_size);
16         }
17     }
18
19     printf("After: team size = %d\n", omp_get_num_threads());
20     return 0;
21 }
```

```
gcc -std=c11 -O2 -Wall -Wextra -fopenmp \
    hello_omp.c -o hello_omp
OMP_DYNAMIC=FALSE OMP_NUM_THREADS=4 ./hello_omp
```

One possible output is:

```
Before: team size = 1
Hello from thread 0 of 4
Hello from thread 2 of 4
Hello from thread 1 of 4
Hello from thread 3 of 4
After: team size = 1
```

Read the program

- `omp.h` declares OpenMP runtime functions.
- `omp_get_thread_num()` returns an ID within the current team, from 0 to the team size minus one.
- `omp_get_num_threads()` reports the size of the current team; outside a parallel region in this example it returns 1.
- Variables declared inside the block, such as `tid`, are local to each thread's execution of the block.
- `critical` lets only one thread at a time execute the print block. It does not force threads to print in numerical order.

Your tasks

1. Run with 1, 2, and 4 threads. Record the requested and actual team sizes.
2. Repeat a run three times. Does the order of the greeting lines change?
3. Add a print statement immediately before the parallel region and one inside it. Explain the difference in the number of times they execute.
4. Explain why “After” appears only after all greeting lines.

Correctness check: each actual thread prints one greeting; thread IDs are valid for the reported team. Different greeting orders are expected.

4 Exercise 2: Understand shared and private data

Goal: choose whether threads access a common object or separate copies.

Clause	Meaning in these C examples
<code>shared(x)</code>	Threads refer to the same object. Concurrent updates may need synchronization.
<code>private(x)</code>	Each thread receives a separate object. An ordinary scalar copy is not initialized from the original value; assign it before reading it.
<code>firstprivate(x)</code>	Each private copy starts with the original value at entry to the construct.
<code>default(none)</code>	Require explicit attributes for variables whose sharing is not predetermined. This helps expose unintended sharing.

`firstprivate` initializes the private copy; it does not copy the final value back to the original object [4].

Listing 3: Save as `data_sharing.c`

```

1  #include <stdio.h>
2  #include <omp.h>
3
4  int main(void)
5  {
6      int base = 10;
7      int scratch = -1;
8
9      #pragma omp parallel default(none) \
10         firstprivate(base) private(scratch)
11     {
12         int tid = omp_get_thread_num();
13         scratch = tid * tid;    /* Initialize before reading. */
14         base += tid;           /* Changes this private copy. */
15
16         #pragma omp critical
17         {
18             printf("thread=%d base=%d scratch=%d\n",
19                 tid, base, scratch);
20         }
21     }
22
23     printf("Outside: base=%d scratch=%d\n", base, scratch);
24     return 0;
25 }
```

```

gcc -std=c11 -O2 -Wall -Wextra -fopenmp \
    data_sharing.c -o data_sharing
OMP_NUM_THREADS=4 ./data_sharing
```

Expected values: thread t prints `base=10+t` and `scratch=t2`. The last line prints `base=10` `scratch=-1`. The order of the per-thread lines can vary.

Your tasks

1. Explain why changing `base` inside the region does not change the original `base` after the region.
2. Replace `private(scratch)` with `firstprivate(scratch)`. Before assigning `scratch = tid * tid`, print its initial value safely inside a critical block. What does each thread observe?
3. In the original program, explain why reading `scratch` before its first assignment would be invalid. Do not treat an accidental output value as a meaningful initial value.
4. Remove `firstprivate(base)` while keeping `default(none)`. Compile and explain the diagnostic. Restore the working version afterward.

A private pointer does not create a private array

Copying a pointer copies an address. For example, `firstprivate(a)` can give each thread its own pointer while all those pointers still refer to the same array. Decide who owns the pointed-to memory, not only the pointer variable.

5 Exercise 3: Parallelize vector addition

Goal: distribute independent loop iterations without duplicating work. For vectors a and b , compute $c_i = a_i + b_i$. Iteration i writes only c_i , so different iterations can run concurrently.

Listing 4: Save as `vector_add.c`

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4 int main(void)
5 {
6     const int n = 1000000;
7     double *a = malloc((size_t)n * sizeof(*a));
8     double *b = malloc((size_t)n * sizeof(*b));
9     double *c = malloc((size_t)n * sizeof(*c));
10    if (a == NULL || b == NULL || c == NULL) {
11        fprintf(stderr, "Allocation failed\n");
12        free(a); free(b); free(c);
13        return 1;
14    }
15    for (int i = 0; i < n; ++i) {
16        a[i] = (double)i;
17        b[i] = 2.0 * i;
18    }
19    #pragma omp parallel for default(none) \
20        shared(a, b, c, n) schedule(static)
21    for (int i = 0; i < n; ++i) {
22        c[i] = a[i] + b[i];
23    }
24    int errors = 0;
25    for (int i = 0; i < n; ++i) {
26        if (c[i] != 3.0 * i) {
27            ++errors;
28        }
29    }
30    printf("c[0]=%.1f c[n-1]=%.1f errors=%d\n",
31        c[0], c[n - 1], errors);
32    free(a); free(b); free(c);
33    return errors != 0;
34 }
```

```
gcc -std=c11 -O2 -Wall -Wextra -fopenmp \  
    vector_add.c -o vector_add  
OMP_NUM_THREADS=4 ./vector_add
```

Expected output:

```
c[0]=0.0 c[n-1]=2999997.0 errors=0
```

The array storage is shared. The loop variable `i` is private by the OpenMP worksharing-loop rules. Each iteration is assigned once, and all iterations finish before the program checks the result. The `static` schedule is a sensible starting point for iterations with similar costs.

The exact comparison is deliberate here: these small integer-valued inputs and results are represented exactly as `double`. General floating-point calculations should use a justified tolerance.

Equivalent structure

The combined directive can also be written as:

Listing 5: Replace the combined loop with this fragment

```
1 #pragma omp parallel default(none) shared(a, b, c, n)  
2 {  
3     #pragma omp for schedule(static)  
4     for (int i = 0; i < n; ++i) {  
5         c[i] = a[i] + b[i];  
6     }  
7 }
```

The separate form is useful when several worksharing operations reuse the same team [3].

Your tasks

1. Verify the result with 1, 2, and 4 threads.
2. Implement $c_i = \alpha a_i + b_i$ with $\alpha = 2$. Add `alpha` to the appropriate clause and change the expected result to $4i$.
3. Replace the combined directive with the equivalent structure above. Confirm that the numerical result stays the same.
4. Explain what happens if the `omp for` line is removed from that structure. Why can identical concurrent writes still constitute a data race?

Check dependencies before adding a directive

The loop `a[i] = a[i-1] + b[i]` has a dependency between adjacent iterations. Adding `parallel for` does not make that dependency disappear. A different parallel algorithm, such as a suitable scan, may be required.

6 Exercise 4: Fix accumulation with a reduction

Goal: understand a data race and select an appropriate synchronization mechanism.

Why the obvious parallel sum is wrong

Suppose `n` is already defined. This fragment is intentionally incorrect:

Listing 6: Incorrect fragment: unsynchronized shared updates

```
1 long long sum = 0;
2 #pragma omp parallel for shared(sum)
3 for (long long i = 1; i <= n; ++i) {
4     sum += i; /* Data race: multiple threads update sum. */
5 }
```

`sum += i` involves reading and updating shared state. Two threads can read the same old value and overwrite one another's updates. More generally, this is a data race, so the program's behavior is not defined by the model. It may even print the expected answer on some runs. A successful-looking run does not prove correctness, and `volatile` does not repair the race.

Three correct versions

A reduction gives participants private accumulators and combines them at the end. An atomic update protects a supported update of one storage location. A critical region allows one thread at a time to execute a block. For this simple sum, reduction avoids synchronizing every addition [5].

Listing 7: Save as `sum_methods.c`

```
1 #include <stdio.h>
2 #include <omp.h>
3 int main(void)
4 {
5     const long long n = 1000000;
6     const long long expected = n * (n + 1) / 2;
7     long long sum_reduction = 0;
8     long long sum_atomic = 0;
9     long long sum_critical = 0;
10    #pragma omp parallel for default(none) shared(n) \
11        reduction(+:sum_reduction) schedule(static)
12    for (long long i = 1; i <= n; ++i) {
13        sum_reduction += i;
14    }
15    #pragma omp parallel for default(none) \
16        shared(n, sum_atomic) schedule(static)
17    for (long long i = 1; i <= n; ++i) {
18        #pragma omp atomic update
19        sum_atomic += i;
20    }
21    #pragma omp parallel for default(none) \
22        shared(n, sum_critical) schedule(static)
23    for (long long i = 1; i <= n; ++i) {
24        #pragma omp critical(sum_guard)
25        {
26            sum_critical += i;
27        }
28    }
29    printf("Expected : %lld\n", expected);
30    printf("Reduction: %lld\n", sum_reduction);
31    printf("Atomic   : %lld\n", sum_atomic);
32    printf("Critical  : %lld\n", sum_critical);
33    return !(sum_reduction == expected &&
34            sum_atomic == expected &&
35            sum_critical == expected);
36 }
```

```
gcc -std=c11 -O2 -Wall -Wextra -fopenmp \
    sum_methods.c -o sum_methods
OMP_NUM_THREADS=4 ./sum_methods
```

All four lines should report 500000500000. The type `long long` is used because this value exceeds the range of a 32-bit signed integer.

Mechanism	Typical use	Main consideration
Reduction	Sum, count, minimum, maximum	Combine independent partial results.
Atomic	One supported shared update	Updates to the same location still contend.
Critical	A multi-statement shared invariant	A large protected block limits concurrency.

All concurrent accesses that conflict must follow a compatible synchronization strategy. An atomic update does not automatically make unsynchronized reads elsewhere safe. Critical regions protecting the same invariant must use the same name (or consistently use the unnamed region).

Your tasks

1. Verify all three methods with different thread counts.
2. Explain why `sum_reduction` is in a reduction clause instead of also appearing in `shared(...)` on that directive.
3. Write a parallel sum of squares, $\sum_{i=1}^n i^2$, using `reduction`. Start with $n = 1000$ and verify against $n(n+1)(2n+1)/6$. Use a wide integer type for intermediate arithmetic.
4. After completing Exercise 8, time the three methods with the same input and explain their differences. Do not include printing in the timers.

Floating-point results need a tolerance

Parallel reductions may add values in a different order. Floating-point addition is not associative, so small differences from a serial answer can be normal. Choose a tolerance suitable for the computation; do not assume bitwise equality or dismiss a large error as rounding.

7 Exercise 5: Compare loop schedules

Goal: investigate how scheduling affects work distribution and runtime.

Schedule	Behavior
<code>static</code>	Assign iterations to threads in advance; commonly useful for regular work with little scheduling overhead.
<code>static,chunk</code>	Assign fixed-size chunks cyclically to the team.
<code>dynamic,chunk</code>	Threads request another chunk after finishing their current one; useful when iteration costs vary.
<code>guided,chunk</code>	Use decreasing chunk sizes, subject to a minimum chunk size, apart from a possible smaller final chunk.
<code>runtime</code>	Take the scheduling policy from <code>OMP_SCHEDULE</code> , allowing experiments without recompilation.

Without an explicit chunk size, do not assume an exact portable partition for `static`. `OMP_SCHEDULE` controls loops written with `schedule(runtime)`; it does not override a loop explicitly written with `schedule(static)` [6].

Listing 8: Save as `schedules.c`

```

1  #include <stdio.h>
2  #include <omp.h>
3
4  int main(void)
5  {
6      enum { N = 32 };
7      int owner[N];
8      double value[N];
9      int used = 0;
10     double start = omp_get_wtime();
11
12     #pragma omp parallel default(none) shared(owner, value, used)
13     {
14         #pragma omp single nowait
15         used = omp_get_num_threads();
16
17         #pragma omp for schedule(runtime)
18         for (int i = 0; i < N; ++i) {
19             int work = 100000 * (i + 1);
20             double local = 0.0;
21             for (int k = 1; k <= work; ++k) {
22                 local += 1.0 / k;
23             }
24             value[i] = local;
25             owner[i] = omp_get_thread_num();
26         }
27     }
28     double elapsed = omp_get_wtime() - start;
29
30     double checksum = 0.0;
31     for (int i = 0; i < N; ++i) {
32         checksum += value[i];
33         printf("iteration=%2d thread=%d\n", i, owner[i]);
34     }
35     printf("threads=%d checksum=%.12f seconds=%.6f\n",
36           used, checksum, elapsed);
37     return 0;
38 }
```

```
gcc -std=c11 -O2 -Wall -Wextra -fopenmp \
    schedules.c -o schedules
export OMP_DYNAMIC=FALSE
export OMP_NUM_THREADS=4
OMP_SCHEDULE=static ./schedules
OMP_SCHEDULE=static,1 ./schedules
OMP_SCHEDULE=dynamic,1 ./schedules
OMP_SCHEDULE=guided,1 ./schedules
```

Each iteration writes a separate `owner[i]` and `value[i]`. The work grows with `i`, so a thread receiving only late iterations may do much more work. The timed region excludes the output and checksum calculation. The `single nowait` statement records the actual team size; Exercise 6 explains these directives.

Your tasks

1. Compare the ownership patterns. For `static,1`, relate the pattern to cyclic assignment of one-iteration chunks.
2. Check that the checksum is unchanged for all schedules. In this program, each inner sum and the final checksum retain their serial addition order.
3. Repeat each configuration at least five times and compare median times. If runs are too short, increase the work factor consistently for every policy.
4. Repeat `dynamic` with chunk sizes 1, 4, and 8. Explain the tradeoff between scheduling overhead and distributing uneven work.

Interpretation: a dynamic schedule need not produce a different ownership pattern on every run, and it need not be faster. Measure on your machine; do not infer speed from the printed thread IDs.

8 Exercise 6: Coordinate dependent phases

Goal: understand barriers, `single`, and the consequences of removing synchronization.

This program initializes a vector, computes its mean once, and centers every element around that mean. Each phase depends on the previous phase.

Listing 9: Save as `phases.c`

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 int main(void)
5 {
6     enum { N = 12 };
7     double a[N];
8     double mean = 0.0;
9
10    #pragma omp parallel default(none) shared(a, mean)
11    {
12        #pragma omp for schedule(static)
13        for (int i = 0; i < N; ++i) {
14            a[i] = i + 1.0;
15        } /* Implicit barrier: all elements are initialized. */
16
17        #pragma omp single
18        {
19            double total = 0.0;
20            for (int i = 0; i < N; ++i) {
21                total += a[i];
22            }
23        }
24    }
```

```

23     mean = total / N;
24 } /* Implicit barrier: mean is ready for every thread. */
25
26 #pragma omp for schedule(static)
27 for (int i = 0; i < N; ++i) {
28     a[i] -= mean;
29 }
30 }
31
32 double centered_sum = 0.0;
33 for (int i = 0; i < N; ++i) {
34     centered_sum += a[i];
35 }
36 printf("mean=%.1f first=%.1f last=%.1f sum=%.1f\n",
37        mean, a[0], a[N - 1], centered_sum);
38 return !(mean == 6.5 && a[0] == -5.5 &&
39         a[N - 1] == 5.5 && centered_sum == 0.0);
40 }

```

```

gcc -std=c11 -O2 -Wall -Wextra -fopenmp phases.c -o phases
OMP_NUM_THREADS=4 ./phases

```

Expected output:

```
mean=6.5 first=-5.5 last=5.5 sum=0.0
```

The synchronization rules you used

- `omp for` has an implicit barrier at its end unless `nowait` removes it.
- `single` selects one thread in the team to execute the block; it need not be thread 0. It normally has an implicit barrier at its end. It does not provide an entry barrier.
- `barrier` explicitly makes the team wait until all members reach that point; the synchronization also establishes the required visibility of preceding shared-memory operations.
- The end of the parallel region synchronizes the team before the code after the region continues.

Your tasks

1. Explain which dependency is protected by each of the first two implicit barriers in the program.
2. Change the first loop to `omp for schedule(static) nowait` and add an explicit `#pragma omp barrier` immediately before `single`. Verify that the result remains correct.
3. Explain why removing that explicit barrier would be unsafe. A barrier at the *end* of `single` cannot protect the earlier reads of `a`.
4. Explain why adding `nowait` to `single` is unsafe here unless another synchronization ensures `mean` is ready before it is used.

Every team member must reach a team barrier

Do not put a barrier inside a condition that only some threads enter, such as `if (omp_get_thread_num() == 0)`. Team members must encounter the required barriers and worksharing constructs in a compatible order. Otherwise the program is invalid and may hang.

Optional extension: sections

Use **sections** for a small fixed set of independent blocks. Each section is executed once by one thread in the team. For example, after declaring `left` and `right`, the following fragment computes two results:

```

1 #pragma omp parallel sections default(none) shared(left, right)
2 {
3     #pragma omp section
4     { left = 20 + 1; }
5
6     #pragma omp section
7     { right = 10 * 2; }
8 }
9 /* left and right are ready here. */

```

Two sections do not create useful work for more than two threads at once. For a dynamically generated set of jobs, tasks are often a better fit.

9 Exercise 7: Create independent tasks

Goal: separate creating a piece of work from the thread that executes it.

A **task** creates an explicit unit of work. A team thread may execute it immediately or later. A task does not create a new operating-system thread. This example creates eight tasks inside one **single** region.

Listing 10: Save as tasks.c

```

1 #include <stdio.h>
2 #include <omp.h>
3 int main(void)
4 {
5     enum { N = 8 };
6     int result[N];
7     int owner[N];
8     #pragma omp parallel default(none) shared(result, owner)
9     {
10         #pragma omp single
11         {
12             for (int i = 0; i < N; ++i) {
13                 #pragma omp task default(none) \
14                     firstprivate(i) shared(result, owner)
15                 {
16                     result[i] = i * i;
17                     owner[i] = omp_get_thread_num();
18                 }
19             }
20
21             #pragma omp taskwait
22             for (int i = 0; i < N; ++i) {
23                 printf("task=%d result=%d executed_by=%d\n",
24                     i, result[i], owner[i]);
25             }
26         }
27     }
28
29     for (int i = 0; i < N; ++i) {
30         if (result[i] != i * i) {
31             return 1;
32         }
33     }
34     return 0;
35 }

```

```
gcc -std=c11 -O2 -Wall -Wextra -fopenmp tasks.c -o tasks
OMP_NUM_THREADS=4 ./tasks
```

Expected results: task indices 0 through 7 produce 0, 1, 4, 9, 16, 25, 36, 49. The executing thread IDs are not fixed. The ordered printout is produced by the final loop; it does not reveal execution order.

Why each directive matters

- `single` ensures one thread generates the task set once.
- `firstprivate(i)` captures the iteration's index when the task is generated. Each task keeps the index it needs even if it runs later.
- The shared arrays remain alive until all the tasks complete; each task writes its own element.
- Plain `taskwait` waits for previously generated child tasks of the current task. Here that makes every result ready before printing. It is not a general team barrier or an automatic wait for all descendants of arbitrary tasks [7].

Your tasks

1. Run with 1, 2, and 4 threads. Can one thread execute multiple tasks?
2. Explain why task count and thread count are different quantities.
3. Explain why moving the output loop before `taskwait` is unsafe. The implicit barrier at the end of `single` would occur too late.
4. Extend each task to perform a different amount of numerical work and keep a serial version for validation. Do not use sleep time as a substitute for computational work when studying CPU speedup.

Granularity matters

The square calculations above are intentionally tiny and demonstrate semantics. Creating tasks for such small work usually costs more than doing the arithmetic. Use larger jobs or combine small jobs before expecting a speedup.

10 Exercise 8: Measure numerical correctness and speedup

Goal: compare a serial algorithm with the same computation using OpenMP.

Approximate π with the midpoint rule:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx \approx h \sum_{i=0}^{n-1} \frac{4}{1+x_i^2}, \quad h = \frac{1}{n}, \quad x_i = (i + 0.5)h.$$

The points can be evaluated independently; their contributions are combined with a reduction. This benchmark needs only a few scalar variables.

Listing 11: Save as pi_benchmark.c

```

1  #include <stdio.h>
2  #include <math.h>
3  #include <omp.h>
4  #ifndef NUM_STEPS
5  #define NUM_STEPS 100000000LL
6  #endif
7  int main(void)
8  {
9      const long long n = NUM_STEPS;
10     if (n <= 0) {
11         fprintf(stderr, "NUM_STEPS must be positive\n");
12         return 1;
13     }
14     const double h = 1.0 / (double)n;
15
16     /* Warm up the thread runtime outside both timed intervals. */
17     int warmup_threads = 0;
18     #pragma omp parallel default(none) reduction(+:warmup_threads)
19     {
20         warmup_threads += 1;
21     }
22
23     double serial_sum = 0.0;
24     double start = omp_get_wtime();
25     for (long long i = 0; i < n; ++i) {
26         double x = (i + 0.5) * h;
27         serial_sum += 4.0 / (1.0 + x * x);
28     }
29     double pi_serial = h * serial_sum;
30     double serial_seconds = omp_get_wtime() - start;
31
32     double parallel_sum = 0.0;
33     int used = 0;
34     start = omp_get_wtime();
35     #pragma omp parallel default(none) shared(n, h, used) \
36         reduction(+:parallel_sum)
37     {
38         #pragma omp single nowait
39         used = omp_get_num_threads();
40
41         #pragma omp for schedule(static)
42         for (long long i = 0; i < n; ++i) {
43             double x = (i + 0.5) * h;
44             parallel_sum += 4.0 / (1.0 + x * x);
45         }
46     }
47     double pi_parallel = h * parallel_sum;
48     double parallel_seconds = omp_get_wtime() - start;
49
50     double difference = fabs(pi_serial - pi_parallel);
51     double reference_error = fabs(pi_parallel - acos(-1.0));
52     printf("n=%lld warmup_threads=%d actual_threads=%d\n",
53           n, warmup_threads, used);
54     printf("serial_pi=%.12f parallel_pi=%.12f\n",
55           pi_serial, pi_parallel);
56     printf("difference=%.3e reference_error=%.3e\n",
57           difference, reference_error);
58     printf("serial_seconds=%.6f parallel_seconds=%.6f\n",
59           serial_seconds, parallel_seconds);
60     if (parallel_seconds > 0.0) {
61         double speedup = serial_seconds / parallel_seconds;
62         printf("speedup=%.3f efficiency=%.3f\n",
63               speedup, speedup / used);
64     }
65     return difference > 1e-8;
66 }

```

```
gcc -std=c11 -O2 -Wall -Wextra -fopenmp \
    pi_benchmark.c -o pi_benchmark -lm
export OMP_DYNAMIC=FALSE
OMP_NUM_THREADS=1 ./pi_benchmark
OMP_NUM_THREADS=2 ./pi_benchmark
OMP_NUM_THREADS=4 ./pi_benchmark

# Optional: change the size while compiling, without editing the file.
gcc -std=c11 -O2 -Wall -Wextra -fopenmp \
    -DNUM_STEPS=2000000000LL pi_benchmark.c -o pi_benchmark -lm
```

Validate before interpreting speed

For the supplied large n , both estimates should be close to 3.141592653589793. The program reports two different checks:

- **difference**: agreement between serial and parallel implementations.
- **reference_error**: accuracy relative to the library's value of π , including both integration and floating-point error.

The exit status checks serial-parallel agreement against 10^{-8} , a practical tolerance for this exercise's recommended sizes. If you change the algorithm or greatly change the input size, reconsider the tolerance. Matching implementations can share the same error, so also inspect the reference error.

Understand the timer

`omp_get_wtime()` returns elapsed wall-clock time in seconds. Both reads for a duration are made by the same thread [8]. The timed parallel interval includes team entry, loop execution, synchronization, and reduction. It excludes the warm-up and printed output. It is a kernel benchmark, not an end-to-end application benchmark.

Let T_s be the serial time and T_p the parallel time using p actual threads for the same input:

$$S_p = \frac{T_s}{T_p}, \quad E_p = \frac{S_p}{p}.$$

$S_p > 1$ means the parallel version is faster. Efficiency is a ratio; multiply by 100 to express it as a percentage. A serial loop and an OpenMP region with one thread are different baselines: the latter still has OpenMP overhead.

Your measurement procedure

1. Record CPU model, logical CPU count, VM vCPU count if applicable, compiler version, flags, and `NUM_STEPS`.
2. Use the same executable and input size for every thread-count comparison. Check the actual thread count reported by the timed region.
3. Run each configuration at least five times while avoiding other heavy workloads. If either timer is too short for stable measurement, increase `NUM_STEPS` consistently until times are comfortably measurable, for example roughly 0.1 seconds or longer.
4. Keep all raw times. Use the median serial time and median parallel time in the corresponding row to calculate the reported speedup. If serial times drift substantially, rerun with a more stable workload and vary run order.
5. Plot thread count against runtime and speedup. Explain both trends.

Actual p	Median T_s (s)	Median T_p (s)	S_p	E_p
1				
2				
4				
Other				

Only include thread counts you actually tested. These cells are a worksheet, not benchmark results.

Explain the limits

If a fraction f of execution is inherently serial, idealized Amdahl scaling gives

$$S_p \leq \frac{1}{f + (1 - f)/p}.$$

This bound assumes fixed work and neglects added parallel overhead. Real programs also encounter memory bandwidth limits, load imbalance, synchronization, CPU frequency changes, and VM interference. More threads therefore need not reduce runtime.

Your final questions:

1. At which tested thread count was the program fastest?
2. Does one OpenMP thread match the time of the serial loop? Why might it not?
3. How does increasing the problem size change the usefulness of parallelism?
4. What additional measurements would help explain a slowdown?

11 Optional mini-project: Parallel matrix multiplication

Use the ideas above to compute $C = AB$ for dense $N \times N$ matrices:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} B_{kj}.$$

Store each matrix in a one-dimensional `double` array using row-major indexing, `a[i * n + j]`. Allocate matrices on the heap, check allocations, and start with $n = 128$ or 256 .

Listing 12: Kernel fragment; `a` and `b` must already be initialized

```

1 #pragma omp parallel for default(none) \
2   shared(a, b, c, n) schedule(static)
3 for (int i = 0; i < n; ++i) {
4     for (int j = 0; j < n; ++j) {
5         double local = 0.0;
6         for (int k = 0; k < n; ++k) {
7             local += a[i * n + k] * b[k * n + j];
8         }
9         c[i * n + j] = local;
10    }
11 }
```

The outer loop assigns different rows of C . `local` belongs to one execution of the inner body, so no cross-thread reduction is needed for it. Do not parallelize the `k` loop by simply allowing several threads to update the same `c[i * n + j]` without a correct accumulation strategy.

1. Build a complete serial program and a parallel version using the same loop order and compiler flags.
2. First set B to the identity matrix and check that $C = A$. Then use nontrivial, deterministic values in both matrices and compare all outputs with the serial implementation using a stated tolerance.
3. Time only the multiplication in both versions. Record initialization and verification separately if reporting end-to-end time.
4. Repeat for several matrix sizes and thread counts.
5. As an extension, try `collapse(2)` on the `i` and `j` loops and measure the effect. It distributes the combined (i, j) iteration space; each output still has its own local accumulation.

Keep comparisons interpretable

If you change loop order, tiling, or data layout, apply the equivalent change to the serial baseline when isolating thread speedup. These optimizations can matter as much as the number of threads, especially for memory-intensive work.

12 Troubleshooting and good practice

Symptom	What to check
Undefined reference to an <code>omp_...</code> function	Include <code>-fopenmp</code> when linking. If compiling and linking separately, use it in both steps.
<code>omp.h</code> cannot be found	Confirm you are using the installed GCC C compiler and its development files. These commands target GCC on Linux.
Only one greeting appears	Check the compilation flags, the parallel directive, <code>OMP_NUM_THREADS</code> , and any thread limits. Query the team size inside the parallel region, not only outside it.
Greeting lines appear in an unexpected order	Thread scheduling is not deterministic. Critical sections serialize access but do not sort thread IDs.
Results change incorrectly with thread count	Inspect shared writes, uninitialized private variables, missing barriers, index ranges, and task data lifetimes.
Small floating-point differences	Check a justified tolerance and the mathematical error; reduction order can affect rounding.
The program hangs	Check that all required team members encounter each barrier and worksharing construct in a compatible order. Also check loop termination.
More threads are slower	Check problem size, contention, memory bandwidth, oversubscription, VM limits, and whether output is inside the timed section.
Changing <code>OMP_SCHEDULE</code> has no effect	Verify that the loop uses <code>schedule(runtime)</code> rather than a fixed schedule.

12.1 A practical review checklist

1. **Ownership:** for each shared object, identify who reads and writes it. Distinct array elements can be independently writable; a shared scalar accumulation normally requires coordination.
2. **Initialization:** initialize private variables before use. Use `firstprivate` only when an initial value is actually needed.
3. **Dependencies:** prove required producer work completes before consumers access its result. Use `nowait` only with that proof.
4. **Validation:** compare all relevant outputs with a serial result or an independent reference, not just a few printed elements.
5. **Measurement:** keep input, flags, and timed work consistent. Save raw runs and report actual team sizes.

13 Suggested lab report

Use the following structure to document your work. Your instructor may adapt the submission format and grading policy.

1. **Environment:** machine or VM configuration, OS, GCC version, compiler flags, and tested thread counts.
2. **Exercises 1–7:** source filenames, representative output, correctness checks, modifications, and answers to the investigation questions.
3. **Exercise 8:** raw measurements, the summary table, runtime and speedup plots, numerical errors, and an explanation of the observed scaling.
4. **Reflection:** one bug you encountered, how you diagnosed it, and which OpenMP mechanism resolved it.
5. **Reproduction:** exact compilation and execution commands, with source code included or linked in a version-controlled repository.

Quick reference

Directive or function	Purpose
<code>#pragma omp parallel</code>	Execute a region using a team.
<code>#pragma omp for</code>	Share loop iterations among the current team.
<code>#pragma omp parallel for</code>	Create a team and distribute a loop.
<code>reduction(+:sum)</code>	Accumulate partial sums safely.
<code>#pragma omp atomic update</code>	Protect a supported single-location update.
<code>#pragma omp critical(name)</code>	Permit one thread at a time in a named region.
<code>#pragma omp single</code>	Execute a block once by one team member.
<code>#pragma omp barrier</code>	Synchronize the team.
<code>#pragma omp task</code>	Generate a task for the team to execute.
<code>#pragma omp taskwait</code>	Wait for prior child tasks of the current task.
<code>omp_get_thread_num()</code>	Get the calling thread's ID within its team.
<code>omp_get_num_threads()</code>	Get the current team size.
<code>omp_get_wtime()</code>	Read a wall-clock timer in seconds.

References

- [1] GNU Compiler Collection. *OpenMP: using GCC*. <https://gcc.gnu.org/onlinedocs/gcc/OpenMP.html>.
- [2] OpenMP Architecture Review Board. *OpenMP 5.2, parallel construct*. <https://www.openmp.org/spec-html/5.2/openmpse57.html>.
- [3] Lawrence Livermore National Laboratory. *OpenMP work-sharing constructs*. https://hpc-tutorials.llnl.gov/openmp/work_sharing_constructs/.
- [4] OpenMP Architecture Review Board. *OpenMP 5.2, firstprivate clause*. <https://www.openmp.org/spec-html/5.2/openmpsu38.html>.
- [5] OpenMP Architecture Review Board. *OpenMP 5.1, reduction clauses and directives*. <https://www.openmp.org/spec-html/5.1/openmpsu117.html>.
- [6] OpenMP Architecture Review Board. *OpenMP 5.1, OMP_SCHEDULE environment variable*. <https://www.openmp.org/spec-html/5.1/openmpse58.html>.
- [7] OpenMP Architecture Review Board. *OpenMP 5.2, taskwait construct*. <https://www.openmp.org/spec-html/5.2/openmpse93.html>.
- [8] GNU libgomp. *omp_get_wtime: elapsed wall-clock time*. https://gcc.gnu.org/onlinedocs/libgomp/omp_005fget_005fwtime.html.
- [9] Lawrence Livermore National Laboratory. *OpenMP tutorial*. <https://hpc-tutorials.llnl.gov/openmp/>.

The references document language and runtime semantics. The programs and exercise instructions in this tutorial are self-contained teaching examples. The selected constructs are established CPU OpenMP features; the version numbers in the reference URLs identify the cited documentation, not a requirement to use every feature of those specification versions.